

Conectivas Básicas

Sean p , q proposiciones (verdadero o falso).

Conectiva	Símbolo	Nombre
Negación	$\neg p$	No
Conjunción	$p \wedge q$	Y (AND)
Disyunción	$p \vee q$	O (OR)
Implicación	$p \rightarrow q$	Si...entonces
Doble implicación	$p \leftrightarrow q$	Si y solo si

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

Conectivas en Programación

Equivalencias CS

- $p \wedge q == p \ \&\& \ q$ (ambos ciertos)
- $p \vee q == p \ || \ q$ (al menos uno)
- $\neg p == !p$ (inversión)

Ejemplo: condición en código

```
if (x > 0 && y > 0) { ... }
```

Equivalencias CS

- $p \wedge q == p \ \&\& \ q$ (ambos ciertos)
- $p \vee q == p \ || \ q$ (al menos uno)
- $\neg p == !p$ (inversión)

Ejemplo: condición en código

```
if (x > 0 && y > 0) { ... }
```

Traducción lógica: $(x > 0) \wedge (y > 0)$ — ambas condiciones deben cumplirse.

Dos proposiciones P y Q son **lógicamente equivalentes** ($P \equiv Q$) si tienen **la misma tabla de verdad**.

Ejemplo clásico — **Leyes de De Morgan**:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q \quad \neg(p \vee q) \equiv \neg p \wedge \neg q$$

Leyes Principales

Leyes de Idempotencia

- $p \vee p \equiv p$
- $p \wedge p \equiv p$

Leyes de Identidad

- $p \vee F \equiv p$
- $p \wedge V \equiv p$

Leyes de Dominación

- $p \vee V \equiv V$
- $p \wedge F \equiv F$

Leyes de Doble Negación

- $\neg\neg p \equiv p$

Leyes Conmutativas

- $p \vee q \equiv q \vee p$
- $p \wedge q \equiv q \wedge p$

Leyes Asociativas

- $(p \vee q) \vee r \equiv p \vee (q \vee r)$
- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$

Leyes Distributivas

- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Leyes de De Morgan

- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
- $\neg(p \vee q) \equiv \neg p \wedge \neg q$

Aplicación: Simplificación

Problema: Demostrar que $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Aplicación: Simplificación

Problema: Demostrar que $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Usando la ley distributiva **directamente**:

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r) \quad \text{Ley distributiva}$$

Aplicación: Simplificación

Problema: Demostrar que $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Usando la ley distributiva **directamente**:

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r) \quad \text{Ley distributiva}$$

Ejemplo en código: simplificar `if (x || (y && z))`

Equivale a: `if ((x || y) && (x || z))` — a veces más eficiente.

La Implicación ($p \rightarrow q$)

$p \rightarrow q$ es **falsa** solo cuando p es V y q es F.

Terminología

p — **Antecedente** (hipótesis, condición)

q — **Consecuente** (tesis, conclusión)

Equivalencias útiles

$$p \rightarrow q \equiv \neg p \vee q \equiv \neg q \rightarrow \neg p$$

La segunda se llama **contron recíproco** — útil en demostraciones.

Reglas de Inferencia

Una inferencia es válida si siempre que las premisas son V, la conclusión también es V.

Nombre	Forma	Intuición
Modus Ponens (MP)	$p \rightarrow q, p \vdash q$	Si p implica q , y p es V, entonces q es V
Modus Tollens (MT)	$p \rightarrow q, \neg q \vdash \neg p$	Si q es F, entonces p debe ser F
Silogismo Hipotético (SH)	$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$	Cadena de implicaciones
Silogismo Disyuntivo (SD)	$p \vee q, \neg p \vdash q$	Si una opción es falsa, la otra es V

Ejemplo: Modus Ponens en Código

Premisa 1 (invariante):

```
if (x != null) then x.length >= 0
```

Premisa 2 (condición):

```
x != null es true
```

Conclusión (por MP):

```
x.length >= 0
```

Ejemplo: Modus Ponens en Código

Premisa 1 (invariante):

```
if (x != null) then x.length >= 0
```

Premisa 2 (condición):

```
x != null es true
```

Conclusión (por MP):

```
x.length >= 0
```

Esto es exactamente lo que hace un compilador/verificador:

- 1 Conocer una precondition ($p \rightarrow q$)
- 2 Comprobar que la precondition se cumple (p)
- 3 **Inferir** la postcondition (q) automáticamente

Afirmación del consecuente (FALSA):

$$p \rightarrow q, q \not\models p \quad \text{¡INCORRECTO!}$$

Ejemplo: “Si llueve, el suelo está mojado. El suelo está mojado. $\Rightarrow$ ¿Está lloviendo?”

Afirmación del consecuente (FALSA):

$$p \rightarrow q, q \not\vdash p \quad \text{¡INCORRECTO!}$$

Ejemplo: “Si llueve, el suelo está mojado. El suelo está mojado. $\Rightarrow$ ¿Está lloviendo?”

Negación del antecedente (FALSA):

$$p \rightarrow q, \neg p \not\vdash \neg q \quad \text{¡INCORRECTO!}$$

Ejemplo: “Si estudias, apruebas. No estudiaste. $\Rightarrow$ ¿Aprobaste?”

Cuantificador Universal ($\forall$) y Existencial ($\exists$)

Universal: $\forall x, P(x)$

“Para **todo** x , $P(x)$ es verdadero.”

Ejemplo: $\forall n \in \mathbb{N}, n^2 \geq 0$ (todo entero al cuadrado es no negativo)

Existencial: $\exists x, P(x)$

“Existe **al menos un** x tal que $P(x)$ es verdadero.”

Ejemplo: $\exists n \in \mathbb{N}, n^2 = 4$ (existe $n = 2$)

Negación:

$$\neg(\forall x, P(x)) \equiv \exists x, \neg P(x) \quad \neg(\exists x, P(x)) \equiv \forall x, \neg P(x)$$

Cuantificadores Restringidos

En CS solemos cuantificar sobre un dominio específico:

$\forall n \in \mathbb{Z}, n > 0 \rightarrow n^2 > 0$ “Todo entero positivo tiene cuadrado positivo”

$\exists x \in \mathbb{R}, x^2 = 2$ “Existe una raíz cuadrada de 2”

En código:

- `for all i: assert(condition(i))` $\equiv \forall i, P(i)$
- `if (exists i: condition(i))` $\equiv \exists i, P(i)$

Orden de los Cuantificadores ¡IMPORTANTE!

El orden **sí importa** cuando hay cuantificadores distintos:

$\forall x \exists y, (x < y)$ “Todo x tiene un y mayor” = VERDADERO

$\exists y \forall x, (x < y)$ “Existe un y mayor que todo x ” = FALSO

Regla: $\forall x \forall y$ y $\exists x \exists y$ son conmutativos; con distintos, el orden cambia el significado.

Definiciones como Bicondicionales

En matemáticas, una **definición** es siempre un “si y solo si”:

$$\text{“}n \text{ es par”} \iff \exists k \in \mathbb{Z}, n = 2k$$

$$\text{“}x \text{ es primo”} \iff x > 1 \wedge \forall a, b \in \mathbb{Z}^+, (a|x \wedge b|x) \rightarrow (a = 1 \vee b = 1)$$

Parafraseando: para **demostrar** que un número es par, basta exhibir el k . Para **refutar**, encontrar un divisor que no sea 1 ni el propio número.

Estrategias de Demostración

Directa

Assume p , aplica reglas de inferencia, llega a q . Usa modus ponens repetidamente.

Por contrapositivo

$p \rightarrow q \equiv \neg q \rightarrow \neg p$. Demuestra la segunda (a veces más fácil).

Por contradicción

Assume $\neg q$ y p , deduce una contradicción (e.g., F). Entonces $p \rightarrow q$.

Contraejemplo

Para refutar $\forall x, P(x)$, basta un solo x con $\neg P(x)$.

Ejemplo: Demostración Directa

Teorema: Si n es par, entonces n^2 es par.

Demostración:

n es par $\Rightarrow$ por definición, $\exists k \in \mathbb{Z}$, $n = 2k$.

Entonces $n^2 = (2k)^2 = 4k^2 = 2(2k^2)$.

Como $2k^2 \in \mathbb{Z}$, n^2 es par.



Ejemplo: Demostración Directa

Teorema: Si n es par, entonces n^2 es par.

Demostración:

n es par $\Rightarrow$ por definición, $\exists k \in \mathbb{Z}, n = 2k$.

Entonces $n^2 = (2k)^2 = 4k^2 = 2(2k^2)$.

Como $2k^2 \in \mathbb{Z}$, n^2 es par.



¿Ves la estructura?

1. Desempacar definición ($\forall/\exists$)
2. Manipulación algebraica
3. Empaquetar de nuevo en la definición

Resumen: Conectivas

Nombre	Símbolo	Equivale a
Negación	$\neg p$	NOT
Conjunción	$p \wedge q$	p AND q
Disyunción	$p \vee q$	p OR q
Implicación	$p \rightarrow q$	$\neg p \vee q$
Bicondicional	$p \leftrightarrow q$	$(p \rightarrow q) \wedge (q \rightarrow p)$

Resumen: Cuantificadores y Negación

$$\neg \forall \equiv \exists \neg \quad \neg \exists \equiv \forall \neg$$

Ejemplo: “No todos los cisnes son blancos”

$$\neg(\forall x, \text{Cisne}(x) \rightarrow \text{Blanco}(x)) \equiv \exists x, \text{Cisne}(x) \wedge \neg \text{Blanco}(x)$$

Época	Hitos
384–322 a.C.	Aristóteles: silogismos, lógica formal
1847	Boole: <i>Laws of Thought</i> , álgebra booleana
1879	Frege: lógica de primer orden, cuantificadores
1910–13	Russell y Whitehead: <i>Principia Mathematica</i>
1936	Turing: máquina de Turing, lógica $\leftrightarrow$ computación
1940s	Shannon: circuitos lógicos (AND, OR, NOT en hardware)
1960s	Verificación formal, lenguajes de demostración (Coq, ACL2)

¿Por qué esto importa en CS?

Hardware

Puertas lógicas (AND, OR, NOT) $\rightarrow$ circuitos $\rightarrow$ CPU

Software

Pre/poscondiciones, invariantes de ciclo, verificación formal

Algoritmos

Correctitud: “si la entrada satisface P , la salida satisface Q ”

Bases de datos

Consultas SQL = lógica de predicados ($\exists$, $\forall$)

“La lógica es la arquitectura de la computación.”

- **Libro:** Rosen — *Discrete Mathematics and Its Applications*, caps. 1 y 2
- **Libro:** Epp — *Discrete Mathematics with Applications*
- **Online:** <https://proofwiki.org> — base de datos de demostraciones
- **Interactivo:** <https://logicbook.net> — tablas de verdad online